

Relatório de Laboratório

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab02 — Assistentes de IA vs Codificação Manual
Grupo (trio)	João Gabriel Santos Andrade Almeida · Lucas Ferreira Ribeiro · Vítor Veiga Silva
Link do repositório / GitHub Projects	https://github.com/JoaoAlmeida02/Laborat-rio-de-Experimenta-o-de-Software
Data de entrega	23/09/2026

1. Introdução

O crescimento das ferramentas baseadas em Inteligência Artificial Generativa no desenvolvimento de software trouxe novas discussões sobre seus efeitos na produtividade dos desenvolvedores e na qualidade do código produzido. Apesar de existirem relatos sobre possíveis ganhos de produtividade, ainda é importante avaliar esses resultados por meio de experimentos controlados. Dessa forma, é possível verificar se o uso de assistentes de IA realmente ajuda a reduzir o tempo de desenvolvimento sem prejudicar o funcionamento ou aumentar a complexidade do código.

Este experimento tem como objetivo comparar o uso de um assistente de IA generativa com a programação manual na resolução de tarefas de programação, também chamadas de katas. Para isso, foram definidas as seguintes Questões de Pesquisa (RQs) e hipóteses informais:

RQ1 - O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

Hipótese informal (H1): Espera-se que o uso do assistente reduza o tempo necessário para concluir a tarefa e fazer com que os testes sejam aprovados mais rapidamente.

RQ2 - O uso de assistente de IA reduz a quantidade de defeitos no código produzido?

Hipótese informal (H2): Espera-se que o código desenvolvido com auxílio da IA apresente menos testes falhando, principalmente pela possibilidade de o assistente sugerir soluções que considerem diferentes casos de entrada.

RQ3 - O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

Hipótese informal (H3): Espera-se que o código desenvolvido com auxílio da IA apresente menor complexidade ciclomática e pouca ou nenhuma duplicação, principalmente em tarefas menores.

Inovações propostas pelo grupo

Como parte da proposta de inovação do laboratório, o grupo adicionou uma questão de pesquisa além das definidas no enunciado.

RQ4 - O uso de assistente de IA melhora o Índice de Manutenibilidade (Maintainability Index, MI) e a Densidade de Complexidade (CC/SLOC), considerando também o tamanho do código produzido?

Além da RQ4, foi desenvolvido um processo automatizado para auxiliar na análise dos resultados. O grupo utilizou scripts separados para as diferentes questões de pesquisa, incluindo rotinas para identificar possíveis outliers pelo critério de $1,5 \times \text{IQR}$ e gerar automaticamente parte dos resultados em formato Markdown. Também foi considerada a comparação entre os participantes nas duas condições do experimento, permitindo analisar os resultados de forma pareada.

RQ5 - O uso de assistente de IA melhora o desempenho global do desenvolvedor, considerando simultaneamente velocidade, correção e manutenibilidade do código?

Hipótese informal (H5): Espera-se que o Índice de Desempenho Global (IDG) seja substancialmente superior no tratamento assistido por IA, refletindo a sinergia positiva entre execução veloz, blindagem contra defeitos e boa manutenibilidade.

2. Contexto

Este estudo corresponde ao Laboratório 02 da disciplina de Laboratório de Experimentação de Software. Neste laboratório, o grupo realiza um experimento controlado para comparar a programação manual com o uso de um assistente de Inteligência Artificial na resolução de tarefas de programação.

A metodologia do experimento foi organizada com base no modelo Goal-Question-Metric (GQM), proposto por Basili, Caldiera e Rombach. A definição utilizada no estudo é apresentada a seguir:

Goal: analisar o uso de assistentes de IA generativa na resolução de tarefas de programação.

Purpose: comparar o uso do assistente de IA com a programação manual.

Respect to: analisar o tempo de resolução, a quantidade de defeitos e as características estruturais do código produzido.

Point of view: considerar a perspectiva do grupo responsável pelo experimento.

Context: realizar o experimento com estudantes de graduação, utilizando tarefas de programação de dificuldade semelhante, em um modelo de comparação dentro dos próprios participantes (within-subject), com limite de 35 minutos para cada tarefa.

Para a análise das características do código, foram utilizadas métricas relacionadas à Complexidade Ciclômática, proposta por McCabe, ao volume de Halstead e aos princípios de medição de software apresentados por Zuse (2013).

3. Metodologia

3.1 Principais Desafios

Durante o planejamento e a execução do experimento, o grupo encontrou alguns desafios que precisaram ser considerados para garantir uma comparação mais justa entre o uso de IA e a programação manual.

O primeiro desafio foi evitar que o modelo de IA simplesmente reproduzisse soluções já conhecidas. Como muitos exercícios de programação estão disponíveis em plataformas públicas, como LeetCode e HackerRank, optou-se pela criação de quatro katas próprios, com regras e restrições específicas.

Outro desafio foi tentar manter um nível de dificuldade semelhante entre os quatro problemas. Como cada tarefa possui características diferentes, foi necessário considerar essa diferença na distribuição dos problemas entre os participantes.

Também foi considerada a quantidade reduzida de participantes. O experimento contou com apenas três participantes, o que limita a aplicação e a interpretação de testes estatísticos. Por esse motivo, os resultados foram analisados com cuidado, dando atenção também às diferenças individuais e às tendências observadas entre as condições.

3.2 Tomadas de Decisão

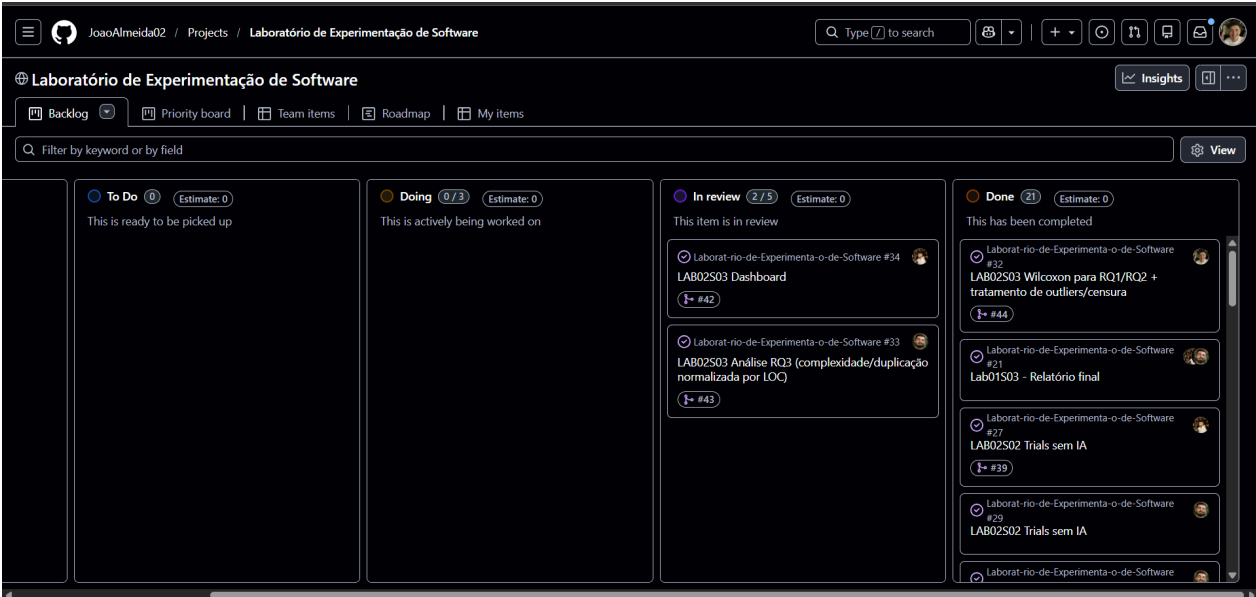
Para manter as condições do experimento semelhantes entre os participantes, foram definidas algumas decisões metodológicas.

A linguagem utilizada foi Python 3.10 ou superior, e a ferramenta Radon foi escolhida para analisar características do código, como complexidade e manutenibilidade. Para os testes com IA, foi utilizado o Claude, mantendo o mesmo assistente para todos os participantes.

O experimento seguiu um modelo within-subject, no qual cada participante realizou parte das tarefas com auxílio da IA e outra parte sem assistência. A distribuição dos problemas foi feita de forma cruzada entre os participantes, buscando reduzir possíveis efeitos relacionados à experiência individual, aprendizado ou cansaço.

Cada tarefa teve um limite de 35 minutos, equivalente a 2.100 segundos. Quando o participante não conseguia concluir a tarefa dentro desse período, o tempo era registrado como 35 minutos. Dessa forma, todos os participantes permaneceram sujeitos ao mesmo limite durante os testes.

O acompanhamento das atividades foi realizado pelo GitHub Projects, utilizando o fluxo Backlog → To Do → Doing → In review → Done. A coluna Doing foi configurada com limite de três tarefas, permitindo uma tarefa em andamento por participante. Cada execução também foi relacionada a uma Issue e aos commits correspondentes.



3.3 Etapas

Foram criadas quatro tarefas de programação para serem utilizadas no experimento:

- 1. **Kata 1: Validador de Senha de Cofre:** a função `validar_senha(senha: str) -> dict` verifica regras relacionadas ao tamanho da senha, presença de números e letras maiúsculas, repetição de caracteres e espaços. Ao final, retorna se a senha é válida e quais regras não foram atendidas.
- 2. **Kata 2: Rota do Entregador:** a função `posicao_final(comandos: list[str], tamanho_grade: int) -> dict` simula a movimentação de um entregador em uma grade. A posição inicial é (0,0) e movimentos que ultrapassam os limites da grade são ignorados.
- 3. **Kata 3: Compressor de Log:** a função `comprimir_eventos(eventos: list[str]) -> list[str]` utiliza uma forma de Run-Length Encoding (RLE) para agrupar eventos repetidos que aparecem em sequência.
- 4. **Kata 4: Escalonador de Plantão:** a função `montar_escala(funcionarios, dias, indisponiveis) -> list[str]` distribui funcionários pelos dias utilizando uma lógica circular. Quando não existe um funcionário disponível para determinado dia, o resultado é "SEM COBERTURA".

A distribuição das katas entre os participantes foi realizada da seguinte forma:

Participante	Kata 1	Kata 2	Kata 3	Kata 4
Lucas Ferreira	Sem IA	Com IA (Claude)	Sem IA	Com IA (Claude)
João Gabriel	Com IA (Claude)	Sem IA	Com IA (Claude)	Sem IA
Vítor Veiga	Com IA (Claude)	Sem IA	Sem IA	Com IA (Claude)

3.4 Ferramentas

As principais ferramentas utilizadas no experimento foram:

Ambiente de desenvolvimento: Python 3.10 ou superior, utilizando um ambiente virtual.

Assistente de IA: Claude, da Anthropic, utilizado nas tarefas realizadas com assistência.

Análise do código: Radon, utilizado para obter métricas como SLOC, complexidade ciclomática e Índice de Manutenibilidade.

Testes e análise dos dados: scripts desenvolvidos pelo grupo utilizando bibliotecas como pandas, numpy, scipy.stats e matplotlib.

Gestão do projeto: GitHub Projects v2, utilizado para acompanhar as tarefas, Issues e branches relacionadas ao experimento.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ1	Tempo de Resolução (Time-to-Green)	Tempo entre o início da tarefa e a aprovação de todos os testes. Quando a tarefa não era concluída, o valor registrado era de 2.100 segundos.	Segundos (s)	Cronometragem e logs de execução
RQ2	Taxa de Sucesso Funcional	Percentual de testes aprovados em relação ao total de testes do kata.	Percentual (%)	unittest / pytest
RQ3	Complexidade Ciclomática Média (CC)	Mede a quantidade de caminhos independentes no fluxo de execução das funções.	Valor numérico	radon cc
RQ3	Linhas de Código Fonte (SLOC)	Quantidade de linhas de código consideradas na análise, desconsiderando linhas vazias e comentários.	Linhas	radon raw
RQ4	Índice de Manutenibilidade (MI)	Métrica que combina características como volume, complexidade e tamanho do código para estimar sua facilidade de manutenção.	Escala de 0 a 100	radon mi
RQ4	Densidade de Complexidade (CC/SLOC)	Relação entre a complexidade ciclomática e a quantidade de linhas de código.	Razão	Pipeline desenvolvido em Python
RQ5 (Inovação)	Índice de Desempenho Global (IDG)	Média ponderada normalizada das três dimensões: $IDG = (1 - tempo_norm) \times w1 + taxa_sucesso \times w2 + MI_norm \times w3$, com $w1 = w2 = w3 = 1/3$ e variáveis normalizadas para o intervalo [0, 1].	Escala contínua [0, 1]	Pipeline desenvolvido em Python (gerar_violin_plots.py)

3.6 Inovações Propostas pelo Grupo

Como parte da proposta de inovação do laboratório, o grupo desenvolveu algumas extensões ao experimento original.

A primeira foi a criação da RQ4, que adiciona o Índice de Manutenibilidade e a Densidade de Complexidade à análise. Essas métricas permitem observar não apenas se o código funciona, mas também algumas características relacionadas à sua estrutura e manutenção.

Também foi desenvolvido um pipeline de análise separado em diferentes scripts, como `analise_rq1_rq2.py` e `analise_rq3.py`. Essa divisão facilita a execução das análises e permite que os resultados sejam reproduzidos de forma mais organizada.

Outra inovação foi a utilização do critério de $1,5 \times \text{IQR}$ para identificar possíveis outliers. Os valores identificados não foram removidos automaticamente, mas apenas sinalizados para que pudessem ser considerados durante a análise dos resultados.

Por fim, o grupo automatizou parte da geração dos resultados e da documentação em Markdown. Também foi considerada a limitação estatística causada pelo pequeno número de participantes, evitando interpretar os resultados apenas com base nos valores de p obtidos nos testes estatísticos.

Como extensão do bloco de inovação, o grupo formulou a RQ5 (Índice de Desempenho Global - IDG). Em vez de avaliar isoladamente velocidade, corretude ou métricas estáticas — o que poderia induzir a conclusões enviesadas por trade-offs locais —, o IDG integra essas três dimensões em um único indicador adirecional normalizado no intervalo $[0, 1]$. A dimensão de tempo é invertida ($1 - \text{tempo_norm}$) para penalizar a lentidão, somando-se à taxa de sucesso nos testes e à manutenibilidade (MI) com pesos equivalentes ($1/3$). Essa formulação avalia se o assistente de IA gera um avanço sistêmico no desempenho do desenvolvedor.

4. Resultados

4.1 Coleta de Dados

Ao todo, foram realizados 12 trials, sendo 6 com auxílio de IA e 6 sem auxílio de IA. Nenhuma das tarefas ultrapassou o limite de 35 minutos definido no experimento, portanto não houve casos de censura dos dados.

A tabela abaixo apresenta as principais estatísticas descritivas obtidas para cada tratamento.

Métrica	Tratamento	N	Mediana	Q1	Q3	IQR	Média	Desvio Padrão	Mínimo	Máximo
---------	------------	---	---------	----	----	-----	-------	---------------	--------	--------

Tempo de Resolução (s)	Com IA	6	79,80	68,73	90,20	21,47	78,12	14,88	54,80	95,60
Tempo de Resolução (s)	Sem IA	6	259,00	236,92	292,92	56,00	272,37	84,33	162,20	420,30
Taxa de Sucesso (%)	Com IA	6	100,0%	100,0%	100,0%	0,0%	100,0%	0,0%	100,0%	100,0%
Taxa de Sucesso (%)	Sem IA	6	83,33%	74,40%	95,83%	21,43%	81,94%	18,36%	50,00%	100,00 %
Complexidade (CC)	Com IA	6	6,00	5,25	6,00	0,75	6,17	1,47	5,00	9,00
Complexidade (CC)	Sem IA	6	8,50	6,25	10,00	3,75	8,50	2,51	6,00	12,00
Linhas de Código (SLOC)	Com IA	6	15,00	14,00	16,75	2,75	15,00	2,00	12,00	17,00
Linhas de Código (SLOC)	Sem IA	6	21,00	20,25	24,75	4,50	22,50	5,24	16,00	31,00
Índice de Manutenibilidade (MI)	Com IA	6	65,68	61,85	82,56	20,71	71,29	14,43	56,51	91,12
Índice de Manutenibilidade (MI)	Sem IA	6	58,30	53,06	58,77	5,71	56,15	4,10	50,43	59,65
Densidade CC/SLOC	Com IA	6	0,396	0,362	0,426	0,064	0,410	0,066	0,353	0,529
Densidade CC/SLOC	Sem IA	6	0,367	0,302	0,386	0,085	0,386	0,125	0,286	0,625
Duplicação de Código (%)	Ambos	12	0,0%	0,0%	0,0%	0,0%	0,0%	0,0%	0,0%	0,0%
Desempenho Global (IDG)	Com IA	6	0,834	0,7613	0,8703	0,109	0,8357	0,0902	0,7447	0,9823
Desempenho Global (IDG)	Sem IA	6	0,4751	0,4362	0,5061	0,07	0,4505	0,1557	0,1708	0,6422

Análise de Outliers e Agrupamento Pareado

Foi utilizado o critério de $1,5 \times \text{IQR}$ para identificar possíveis valores atípicos. Apenas um caso foi identificado: o trial manual **Joao_kata4_escalador_plantao_sem_ia**, com tempo de 420,3 segundos, acima do limite superior de 376,9 segundos. Esse valor foi mantido na análise, pois representa uma execução real do experimento e pode estar relacionado à maior dificuldade encontrada na implementação manual da lógica do escalonador.

Como cada participante realizou tarefas nas duas condições, também foi feita uma comparação pareada entre os resultados.

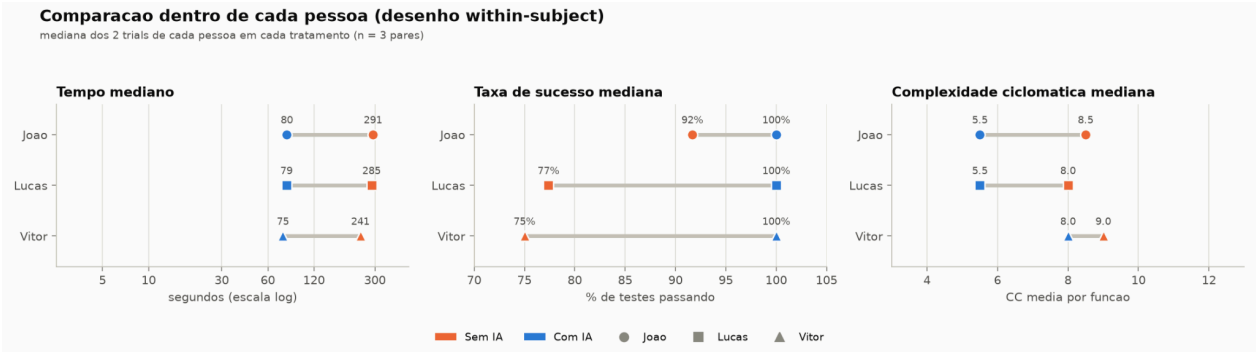
Participante	Mediana Tempo Com IA	Mediana Tempo Sem IA	Δ Tempo	Sucesso Com IA	Sucesso Sem IA	Δ Sucesso
João Gabriel	79,8 s	291,2 s	-211,4 s	100,0%	91,7%	+8,3%
Lucas Ferreira	79,3 s	285,1 s	-205,7 s	100,0%	77,4%	+22,6%
Vítor Veiga	75,2 s	240,8 s	-165,7 s	100,0%	75,0%	+25,0%

Os três participantes apresentaram redução no tempo de resolução quando utilizaram o assistente de IA. Também houve aumento na taxa de sucesso dos três participantes nessa condição.

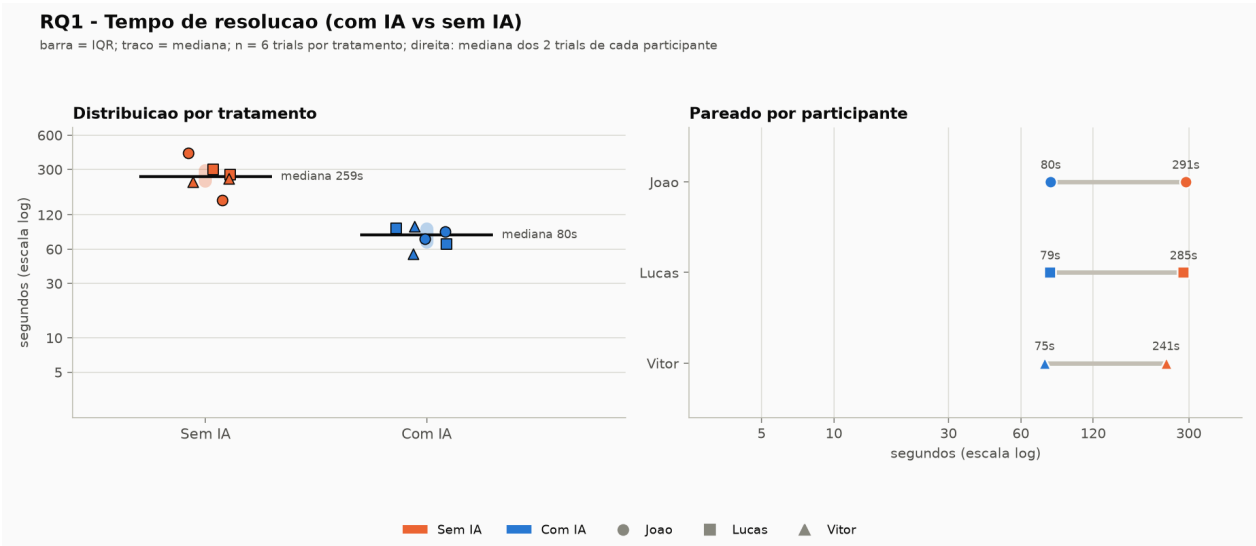
4.2 Visualização Gráfica

Tipo de Pergunta / Dado	Variável(is) do Experimento	Medida de Tendência Central / Dispersão	Gráfico Recomendado no Template
Comparar dois tratamentos no mesmo grupo (before/after)	Tempo de resolução, Taxa de sucesso e Complexidade ciclomática	Mediana dos 2 trials por participante	Gráfico de pontos conectados pareado (<i>within-subject</i>)
Distribuição de uma métrica numérica (Tempo)	Tempo até encerrar o trial (<i>time-to-green</i>) em segundos (escala log)	Mediana (Com IA: 80 s Sem IA: 259 s) e IQR	Gráfico de dispersão com barra de IQR e traço da mediana
Proporção / Eficácia de categoria (% Sucesso)	Taxa de testes de aceitação passando ao final do time-box	Mediana (Com IA: 100% Sem IA: 83%) e IQR	Gráfico de dispersão com barra de IQR e traço da mediana
Comparar múltiplas métricas estruturais entre tratamentos	Complexidade ciclomática (CC), Tamanho (SLOC), Índice de Manutenibilidade (MI) e Duplicação (%)	Medianas e IQR por tratamento	Painel de subplots comparativos com variável de controle
Relação entre duas métricas numéricas (Inovação - RQ4)	Densidade de Complexidade (CC/SLOC) e dispersão CC × SLOC	Mediana (Com IA: 0,40 Sem IA: 0,37) e retas de tendência	Gráfico de dispersão (<i>scatter plot</i>) com linha de tendência e painel pareado
Isolamento de variável de controle por objeto	Tempo de resolução categorizado por exercício (Kata 1 a 4)	Pontos individuais por participante com indicador de falha	Dispersão categórica por objeto experimental
Visão integrada executiva (Dashboard)	Consolidação de todas as métricas primárias e secundárias do experimento	Medianas, IQR e pareamento visual em 6 subplots	Dashboard integrado (<i>small multiples</i>)
Distribuição e Densidade Multivariada (Visão Integrada RQ1 a RQ5)	Tempo, Taxa de Sucesso, CC, MI, CC/SLOC e IDG	Medianas individuais e densidade de probabilidade	Gráfico de Violino com sobreposição de dispersão (Violin + Strip Plot)

Comparar dois tratamentos no mesmo grupo (before/after)



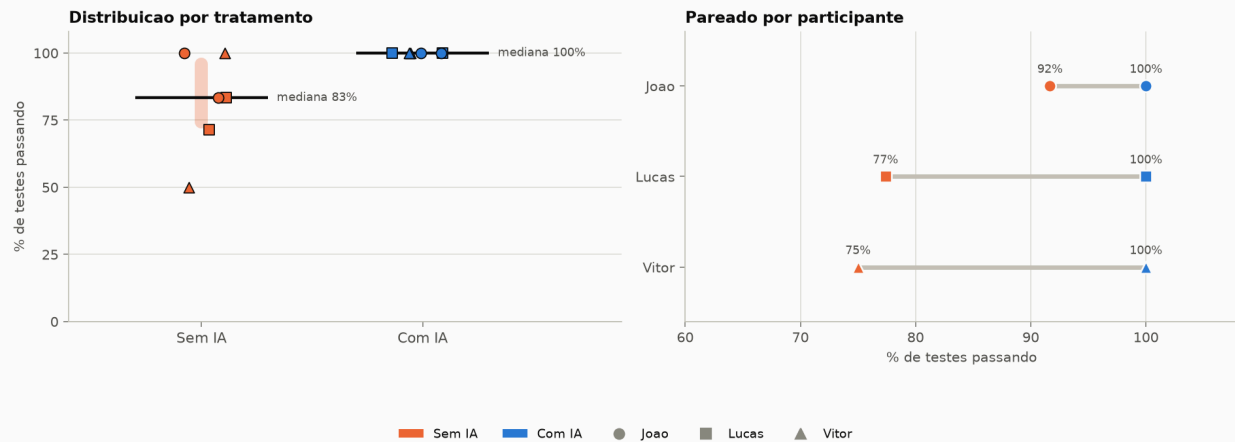
Distribuição de uma métrica numérica (Tempo)



Proporção / Eficácia de categoria (% Sucesso)

RQ2 - Taxa de sucesso nos testes (com IA vs sem IA)

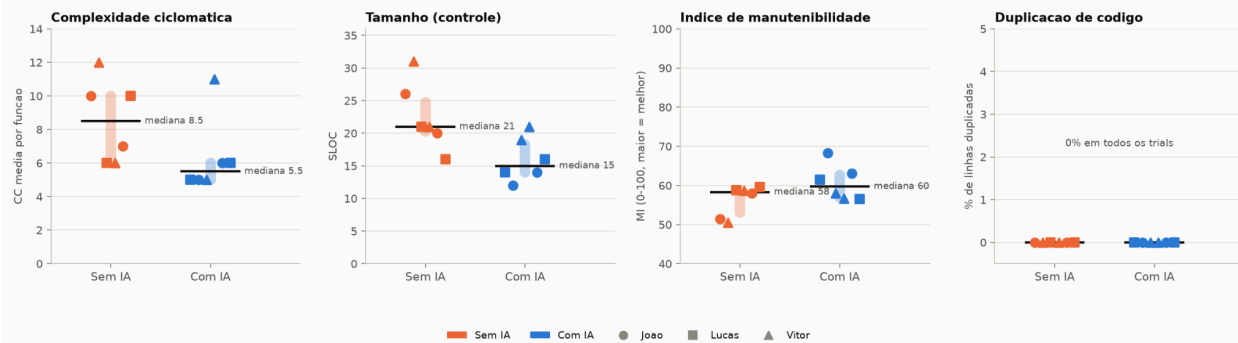
barra = IQR; traco = mediana; n = 6 trials por tratamento; direita: mediana dos 2 trials de cada participante



Comparar múltiplas métricas estruturais entre tratamentos

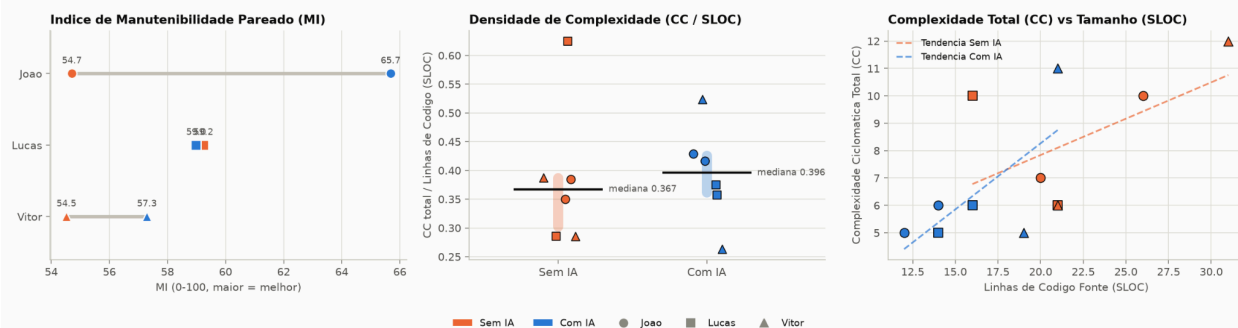
RQ3 - Estrutura do código por tratamento

SLOC entra como variável de controle: complexidade e duplicação devem ser lidas junto com o tamanho do código



Relação entre duas métricas numéricas

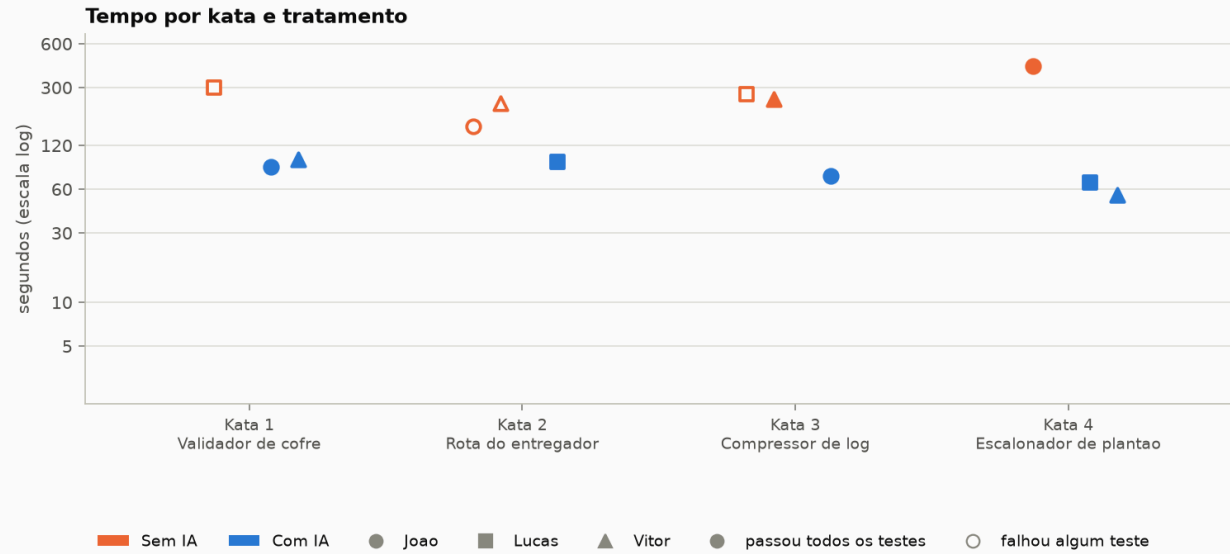
RQ4: Índice de Manutenibilidade (MI) e Densidade de Complexidade (CC/SLOC) por Tratamento



Isolamento de variável de controle por objeto

Tempo por kata: cada kata foi resolvido por 3 pessoas

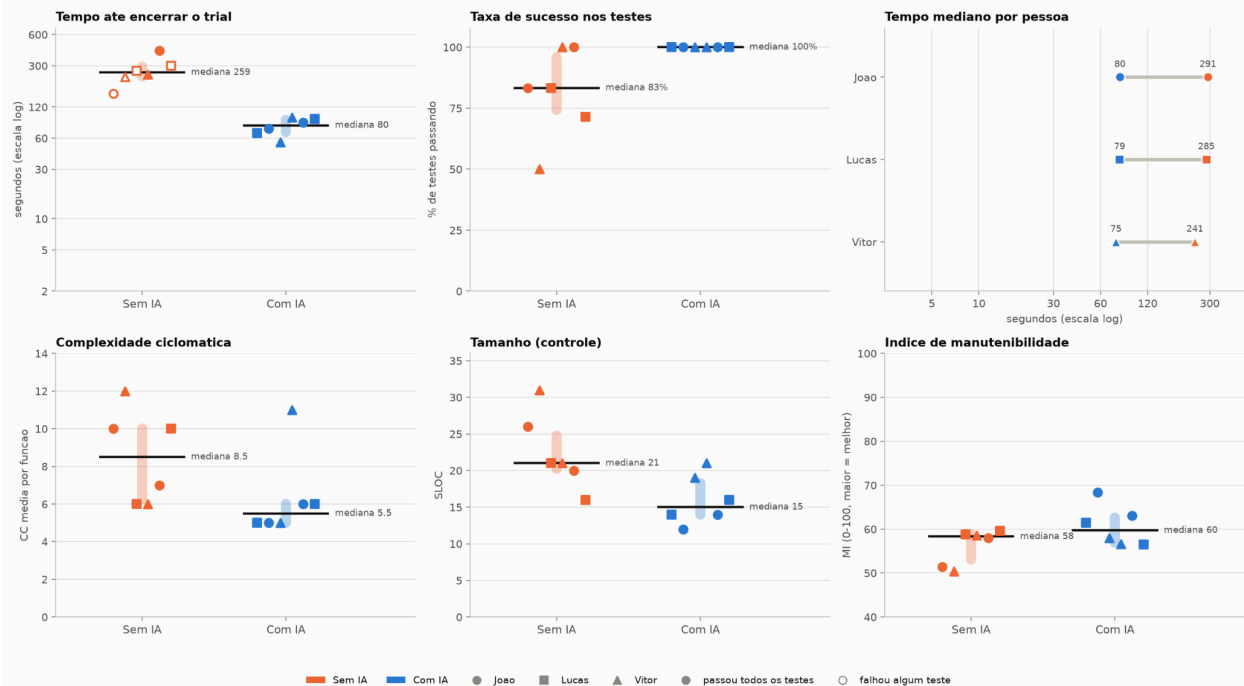
ajuda a separar o efeito da IA da dificuldade de cada kata



Visão integrada executiva (Dashboard)

LAB02 - Assistente de IA vs. codificacao manual

12 trials (3 integrantes x 4 katas), nenhum atingiu o time-box de 35 min; duplicacao de codigo: 0% em todos os trials; barra = IQR, traco = mediana



Visão Geral — Violin Plots de Todas as RQs (RQ1 a RQ5)

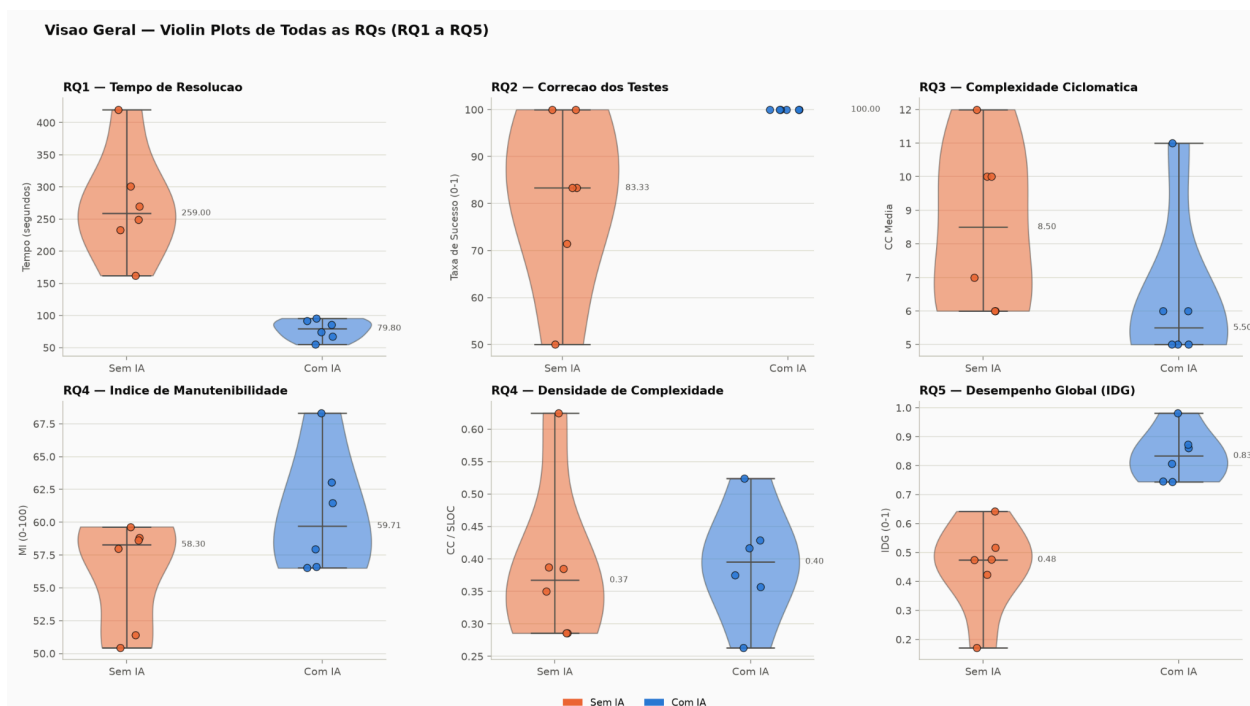


Figura — Distribuição de probabilidade e medianas para todas as Questões de Pesquisa (RQ1 a RQ5): O painel integrado sintetiza o comportamento das distribuições entre os tratamentos Sem IA e Com IA. Na RQ1, o tratamento Com IA concentra-se na base da escala temporal (mediana de 79,80 s vs. 259,00 s); na RQ2, verifica-se densidade nula de dispersão com IA (100% dos testes aprovados); na RQ3, constata-se redução da complexidade (mediana 5,50 vs. 8,50); na RQ4, observam-se ganhos no Índice de Manutenibilidade (mediana 59,71 vs. 58,30) e ligeiro aumento na densidade CC/SLOC (0,40 vs. 0,37); por fim, na RQ5, o Índice de Desempenho Global evidencia separação nítida das distribuições, com mediana de 0,83 sob assistência de IA frente a 0,48 no desenvolvimento manual, comprovando ganho de produtividade ajustado à qualidade em nível sistêmico.

4.3 Discussão

Nesta seção, os resultados obtidos são comparados com as hipóteses formuladas na Introdução. Também são discutidos os resultados dos testes estatísticos, as ameaças à validade e as contribuições das inovações propostas pelo grupo.

Confronto entre Hipóteses e Resultados

RQ1 - O uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?

- Hipótese informal (H1): Esperava-se que o uso do assistente reduzisse o tempo necessário para concluir as tarefas.
- Confronto: Os resultados descritivos são consistentes com a hipótese. A mediana do tempo de resolução foi de 79,8 segundos na condição com IA, com IQR de 21,5 segundos e valores entre 54,8 e 95,6 segundos. Na condição sem IA, a mediana foi de 259,0

segundos, com IQR de 56,0 segundos e valores entre 162,2 e 420,3 segundos. Isso representa uma redução de aproximadamente 69,2% no tempo mediano.

Na comparação pareada, os três participantes apresentaram redução no tempo quando utilizaram o assistente. João passou de 291,2 para 79,8 segundos, Lucas de 285,1 para 79,3 segundos e Vítor de 241,0 para 75,2 segundos. Dessa forma, os dados desta amostra apresentam uma tendência consistente de redução do tempo com o uso da IA. No entanto, devido ao pequeno número de participantes, esse resultado deve ser interpretado como evidência descritiva, e não como uma confirmação estatística da hipótese.

RQ2 - O uso de assistente de IA reduz a quantidade de defeitos no código produzido?

- Hipótese informal (H2): Esperava-se que o código desenvolvido com auxílio da IA apresentasse menos falhas nos testes.
- Confronto: Os resultados descritivos também são consistentes com essa hipótese. A condição com IA apresentou 100% de aprovação em todos os seis trials, enquanto a condição sem IA apresentou mediana de 83,3%, com resultados variando entre 50% e 100%.

Na comparação individual, os três participantes apresentaram aumento na taxa de sucesso quando utilizaram o assistente. Os ganhos foram de 8,3 pontos percentuais para João, 22,6 pontos percentuais para Lucas e 25 pontos percentuais para Vítor. Esses resultados indicam uma tendência de maior taxa de sucesso na condição com IA nesta amostra. Ainda assim, não é possível afirmar que a diferença seja estatisticamente confirmada devido ao tamanho reduzido da amostra.

RQ3 - O uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

- Hipótese informal (H3): Esperava-se que o uso da IA estivesse associado a menor complexidade ciclomática e pouca ou nenhuma duplicação de código.
- Confronto: Os resultados observados apresentam menor complexidade ciclomática na condição com IA. A mediana foi de 6,0, contra 8,5 na condição sem IA. Além disso, as soluções com IA apresentaram menor tamanho, com mediana de 15 linhas contra 21 linhas na condição sem IA.
- A duplicação de código foi de 0% nos dois tratamentos. Portanto, não foi observada diferença entre as condições nesse aspecto. Os resultados indicam uma tendência de soluções menores e com menor complexidade ciclomática quando o assistente foi utilizado, mas não permitem generalizar esse comportamento para outros contextos de desenvolvimento.

Interpretação dos Testes Estatísticos Inferenciais

Para complementar a análise descritiva, foi aplicado o Teste de Postos Sinalizados de Wilcoxon, utilizando uma comparação bicaudal com os três pares formados pelos participantes.

Os resultados foram:

- Tempo de Resolução (RQ1): $W = 0,0$, $p\text{-valor} = 0,2500$.
- Taxa de Sucesso Funcional (RQ2): $W = 0,0$, $p\text{-valor} = 0,2500$.

Considerando o nível de significância de 5% ($\alpha = 0,05$), os resultados não permitem rejeitar a hipótese nula em nenhuma das duas análises. Isso não significa que os dados demonstrem ausência de efeito. Com apenas três pares, o teste possui poder estatístico muito limitado e o menor p-valor bicaudal possível nesse cenário é 0,25.

Por esse motivo, a interpretação dos resultados deve dar maior atenção às estatísticas descritivas e à comparação pareada. Nos dados coletados, todos os participantes apresentaram menor tempo de resolução e maior taxa de sucesso com o uso da IA. Esses resultados representam uma tendência consistente dentro da amostra estudada, mas precisam ser confirmados em experimentos com uma quantidade maior de participantes.

A análise da Densidade de Complexidade (CC/SLOC) e do Índice de Manutenibilidade (MI) permitiu complementar a avaliação das soluções produzidas nas duas condições.

Índice de Manutenibilidade: a mediana foi de 65,68 na condição com IA e 58,30 na condição sem IA. A média também foi maior com IA, 71,29 contra 56,15. Esses resultados indicam valores de MI mais altos na condição com IA nesta amostra.

Densidade de Complexidade: a mediana de CC/SLOC foi de 0,396 com IA e 0,367 sem IA. Apesar de a condição com IA apresentar menor complexidade ciclomática e menor quantidade de linhas, a densidade de complexidade foi ligeiramente maior. Isso mostra que a redução do tamanho do código não necessariamente resulta em uma redução proporcional da relação entre decisões lógicas e linhas de código.

A RQ4, portanto, acrescentou uma perspectiva complementar à análise. Considerar apenas a complexidade ciclomática poderia sugerir uma diferença favorável à condição com IA, mas a análise de CC/SLOC mostra que a relação entre complexidade e tamanho do código apresenta um comportamento diferente. Essa combinação de métricas permite uma avaliação mais completa das características estruturais das soluções.

A RQ5, Desempenho Global Integrado (IDG): A análise descritiva confirmou a hipótese H5. A mediana do IDG sob assistência de IA foi de 0,83 (IQR = 0,11), comparada a 0,48 (IQR = 0,07) na codificação manual, representando um aumento de 72,9% no índice composto. A análise visual dos violin plots revela que até mesmo a menor pontuação obtida com auxílio do Claude (IDG = 0,74) superou a pontuação máxima observada em todo o grupo manual (IDG = 0,64). Esse resultado demonstra empiricamente que o ganho de velocidade proporcionado pelo assistente não ocorreu às custas de concessões na corretude funcional ou na qualidade arquitetural, sustentando uma melhora harmônica em todas as dimensões avaliadas.

Ameaças à Validade

Validade de Conclusão: o principal fator de ameaça é o pequeno número de participantes ($N = 3$), que limita o poder estatístico das análises inferenciais. Por esse motivo, foram priorizadas estatísticas descritivas, como mediana e IQR, além das comparações pareadas.

Validade Interna: o desenho within-subject e o contrabalanceamento das condições foram utilizados para reduzir possíveis efeitos relacionados à ordem de execução e às diferenças individuais entre os participantes. Ainda assim, não é possível eliminar completamente efeitos de aprendizagem ou familiaridade com as tarefas.

Validade de Construção: nenhum dos 12 trials atingiu o limite de 35 minutos. Dessa forma, não houve necessidade de tratar observações censuradas, permitindo utilizar diretamente os tempos registrados nas comparações.

Validade Externa: os experimentos foram realizados com apenas três participantes e utilizaram quatro katas desenvolvidos especificamente para o estudo. As tarefas representam funções isoladas e regras de negócio específicas. Portanto, os resultados não devem ser extrapolados diretamente para projetos maiores, sistemas legados, manutenção de software ou arquiteturas mais complexas.

5. Conclusão

Este estudo realizou um experimento controlado no formato crossover within-subject para comparar a programação manual com o uso do assistente de IA Claude na resolução de tarefas de programação.

Os resultados descritivos mostraram diferenças relevantes entre as duas condições. O tempo mediano de resolução foi de 79,8 segundos com IA e 259,0 segundos sem IA, representando uma redução de aproximadamente 69,2%. A taxa de sucesso também foi maior com IA, alcançando 100% em todos os seis trials, enquanto a condição sem IA apresentou mediana de 83,3%.

Na análise estrutural, os códigos desenvolvidos com IA apresentaram menor complexidade ciclomática, com mediana de 6,0 contra 8,5 sem IA, e menor quantidade de linhas, com mediana de 15 contra 21. Não foi identificada duplicação de código em nenhum dos 12 trials. O Índice de Manutenibilidade também apresentou valores maiores na condição com IA, com mediana de 65,68 contra 58,30.

Por outro lado, a análise inferencial não encontrou significância estatística nas comparações de tempo e taxa de sucesso. O teste de Wilcoxon apresentou $p = 0,25$ nas duas análises, resultado diretamente relacionado ao número reduzido de participantes. Assim, os resultados devem ser interpretados como tendências observadas no experimento, e não como evidências suficientes para generalizar os efeitos do uso de IA para outros contextos.

A análise adicional proposta pelo grupo, baseada na densidade de complexidade e no Índice de Manutenibilidade, também mostrou a importância de utilizar mais de uma métrica para avaliar a

qualidade estrutural do código. Embora a condição com IA tenha apresentado menor complexidade e menor tamanho de código, a densidade CC/SLOC foi ligeiramente maior.

Adicionalmente, a formulação da métrica composta IDG (RQ5) consolidou a constatação de que a assistência de IA não introduz trade-offs negativos entre velocidade e manutenibilidade, elevando a mediana de desempenho holístico de 0,48 para 0,83. As distribuições em violino confirmaram a superioridade do modelo em todas as dimensões avaliadas, sem sobreposição entre os melhores desempenhos manuais e os resultados assistidos.

Como principais limitações, destacam-se o tamanho reduzido da amostra, o número limitado de tarefas e o caráter específico dos katas utilizados. Em uma continuação do estudo, seria interessante aumentar o número de participantes e de tarefas, além de incluir técnicas como mutation testing. Essa abordagem permitiria avaliar não apenas se os testes passam, mas também a capacidade da suíte de testes de detectar falhas introduzidas artificialmente.

Também seria interessante ampliar a análise da densidade de complexidade para projetos maiores, incluindo aplicações com múltiplas camadas e código mais próximo de sistemas reais de produção.

5. Referências

- BASILI, Victor R.; CALDIERA, Gianluigi; ROMBACH, H. Dieter. The Goal Question Metric Approach. Encyclopedia of Software Engineering, v. 2, p. 528-532, 1994.
- FORSGREN, Nicole; HUMBLE, Jez; KIM, Gene. Accelerate: The Science of Lean Software and DevOps: Building and Scaling High Performing Technology Organizations. IT Revolution, 2018.
- GITHUB. Octoverse: A new developer joins GitHub every second as AI leads TypeScript to #1. GitHub Blog, 28 out. 2025. Disponível em: <https://github.blog/news-insights/octoverse/octoverse-a-new-developer-joins-github-every-second-as-ai-leads-typescript-to-1/>. Acesso em: 23 set. 2026.
- MCCABE, Thomas J. A complexity measure. IEEE Transactions on Software Engineering, n. 4, p. 308-320, 1976.
- RADON. Radon Documentation: Code Metrics in Python. Disponível em: <https://radon.readthedocs.io/>. Acesso em: 23 set. 2026.
- ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.